

MLOps Cloud-to-Edge - Stack technique cible

Proposition minimale Fog / Edge / Far Edge - hors sizing GPU

Poste	Fog	Edge	Far Edge
Orchestration	Kubernetes standard	K3s	K3s single-node ou agent léger
Déploiement	Helm + Argo CD / Flux GitOps	GitOps pull quand connecté	
GPU / NVIDIA	GPU Operator, CUDA, drivers	JetPack + NVIDIA Container Runtime	JetPack + TensorRT runtime
Training	Retraining + Finetuning. PyTorch + workflows de training et de fine-tuning.	Annotation (puis Retraining). Inférence possible. Fine-tune léger si possible.	Inférence seule (pas de training).
Serving IA (Inference Server / Runtime to run inference)	Triton Inference Server / MLServer / TorchServe / TensorRT / Runtime custom . Export format ONNX→ONNX Runtime (tests) →build TensorRT Runtime (pour TensorRT runtime de Edge).	Triton si multi-runtimes (NVIDIA, TensorRT, ONNX, PyTorch, TensorFlow); sinon TensorRT Runtime / ONNX Runtime.	TensorRT runtime optimisé.
KServe (Couche K8s de serving ML. Orchestre le serving)	Oui. Expose InferenceService , gérer plusieurs runtimes, standardiser le serving.	Pertinent si Edge gère rollback, sert plusieurs modèles. Utiliser en RawDeployment (plus léger).	Non. Runtime direct : TensorRT, Triton léger.
Registry modèles	MLflow Tracking + Model Registry	Cache MLflow local optionnel	Modèle actif + rollback local
Artefacts / données	S3-compatible : MinIO / S3	Cache objet local ou filesystem structuré	Filesystem local + manifests
DB	PostgreSQL central (stockage <u>metadata</u> MLflow, annotations)	PostgreSQL léger / SQLite (stockage <u>metadata</u> MLflow, annotations)	SQLite
Annotation			Aucune annotation
Flux raw data	Réception + stockage via MinIO/S3 central + Data Mover receiver	NATS JetStream + MinIO replication + cache local + Data Mover custom léger Python/Go.	Data Mover (microservice custom léger Python/Go, wrapper s5cmd/rcclone, MinIO replication) + cache local
Observabilité	Prometheus, Grafana, OpenTelemetry	Grafana Alloy / Prometheus Agent (plus minimal) + buffer local + remote_write vers Fog	métriques inférence
Sécurité			

Lecture : KServe orchestre le serving ; Triton/TensorRT exécutent l'inférence ; NATS JetStream pilote les événements tandis que MinIO/S3 transporte et stocke les objets lourds.

TensorRT Runtime = Runtime d'exécution bas niveau NVIDIA.